Memoization

i member

¿Cuál es el peligro de la Recursión?

Experimentémoslo empíricamente:

```
long fibonacci(int pos) {
    if( pos == 0 ) return 0;
    if( pos == 1 ) return 1;
    return fibonacci(pos - 1) + fibonacci(pos - 2);
}
```

¿Cuál es el peligro de la Recursión?

Experimentémoslo empíricamente:

```
long fibonacci(int pos) {
    if( pos == 0 ) return 0;
    if( pos == 1 ) return 1;
    return fibonacci(pos - 1) + fibonacci(pos - 2);
}
```

A penas 50!!

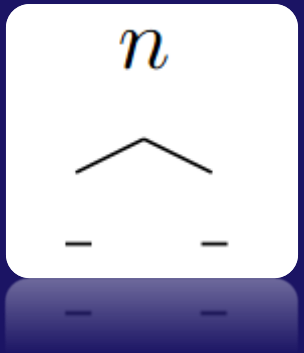
Análisis formal

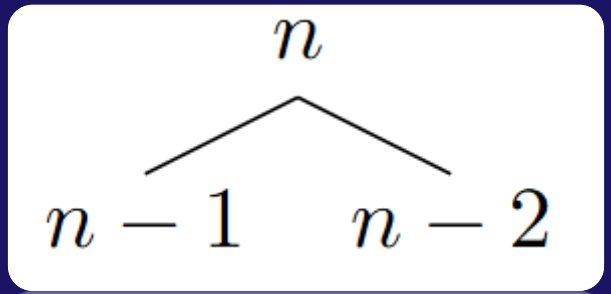
```
long fibonacci(int pos) {
    if( pos == 0 ) return 0;
    if( pos == 1 ) return 1;
    return fibonacci(pos - 1) + fibonacci(pos - 2);
}
```

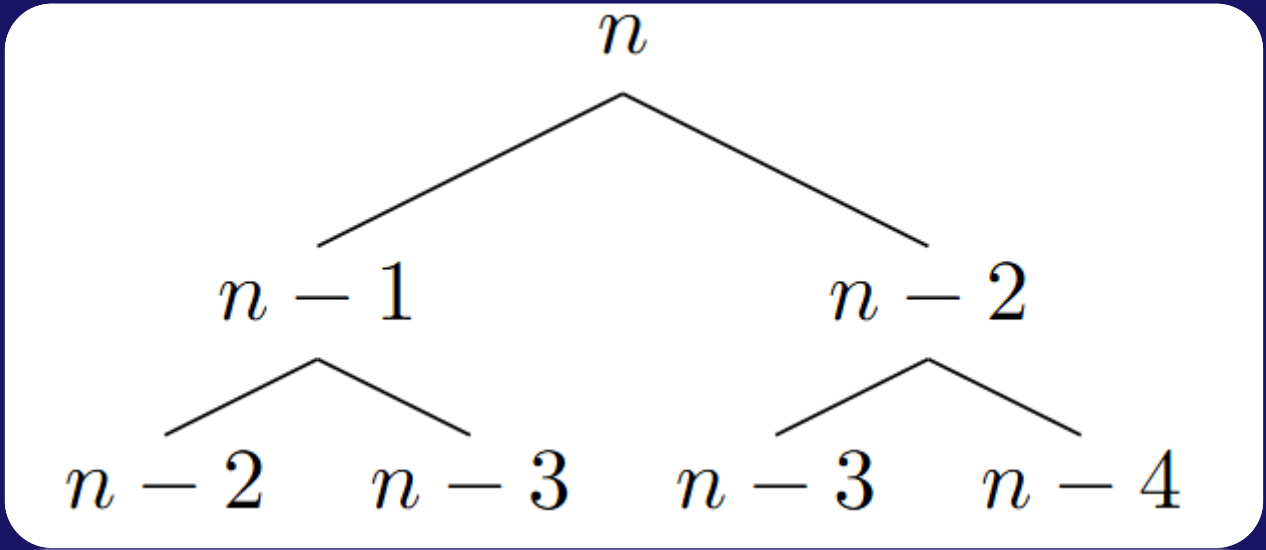
Recurrencia:

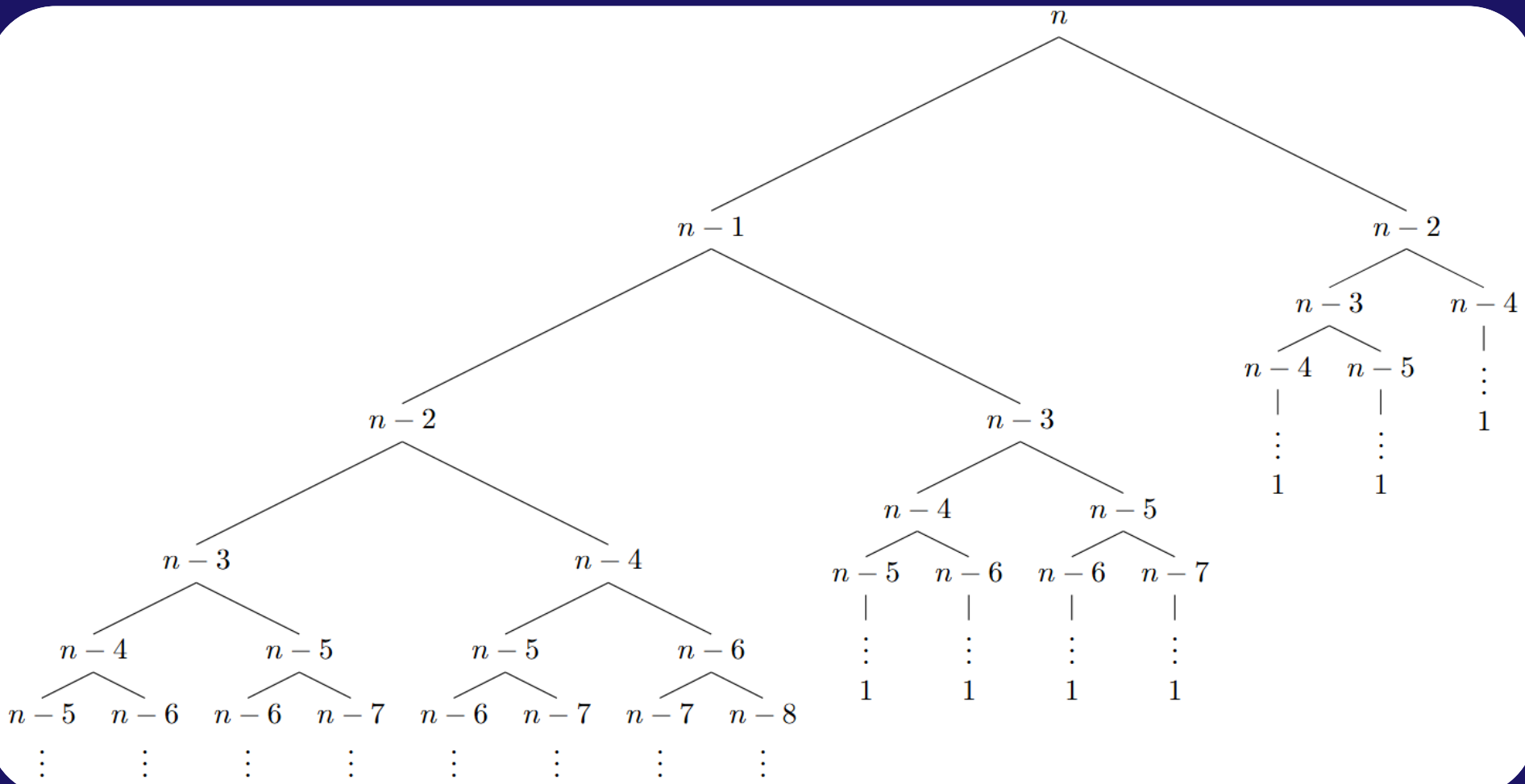
$$T(n) = T(n-1) + T(n-2) + O(1)$$

$$n$$


n


n
$n-1$
$n-2$


n
$n-1$
$n-2$
$n-2$
$n-3$
$n-3$
$n-4$


n
$n-1$
$n-2$
$n-2$
$n-3$
$n-3$
$n-4$
$n-4$
$n-5$
$n-3$
$n-4$
$n-4$
$n-5$
$n-4$
$n-5$
$n-5$
$n-6$
$n-5$
$n-6$
$n-5$
$n-6$
$n-6$
$n-7$
$n-6$
$n-7$
$n-6$
$n-7$
$n-7$
$n-8$
1
1
1
1
1
1
1
1
1

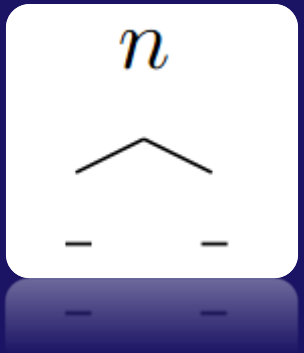
Análisis formal

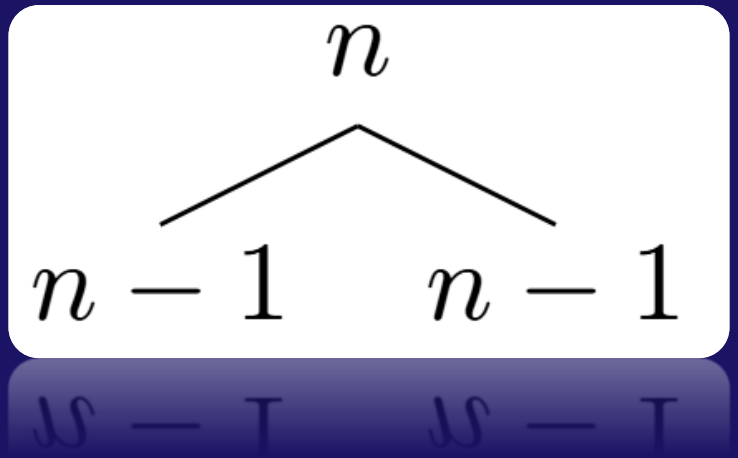
```
long fibonacci(int pos) {
    if( pos == 0 ) return 0;
    if( pos == 1 ) return 1;
    return fibonacci(pos - 1) + fibonacci(pos - 2);
}
```

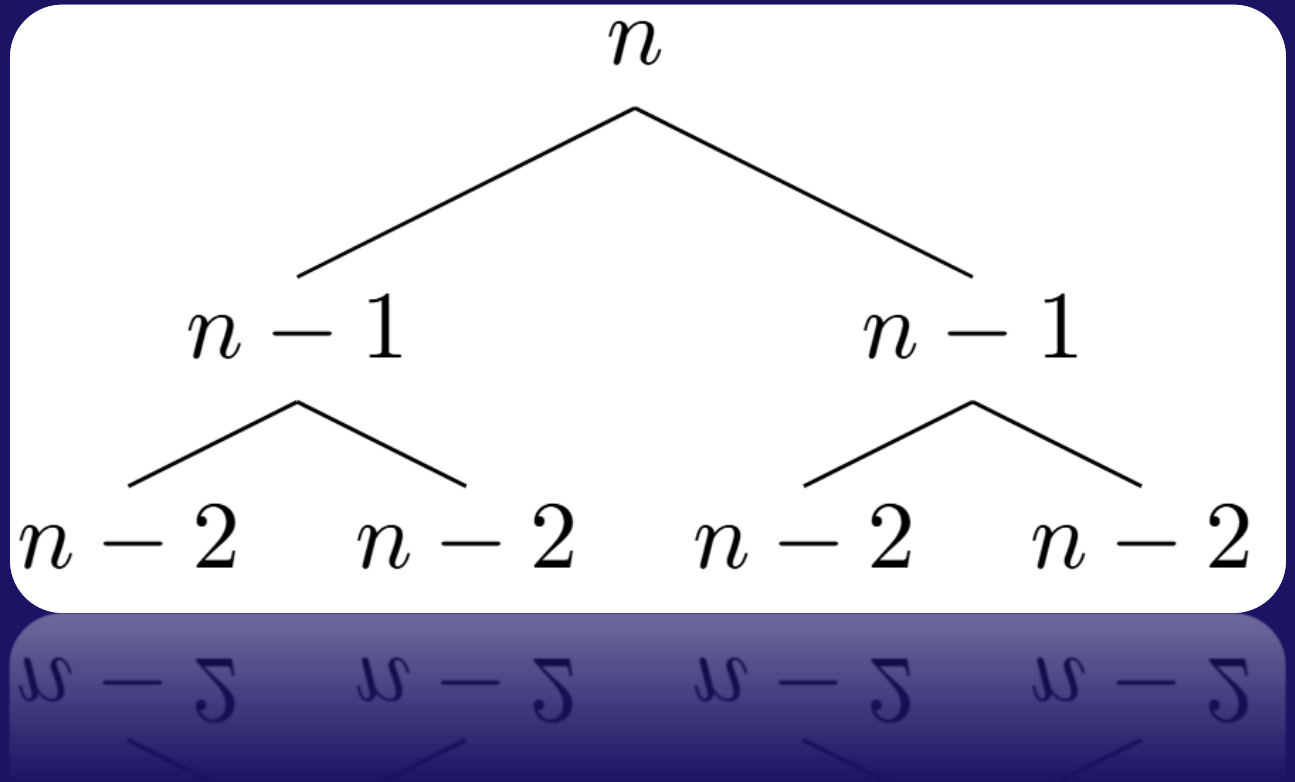
Recurrencia:

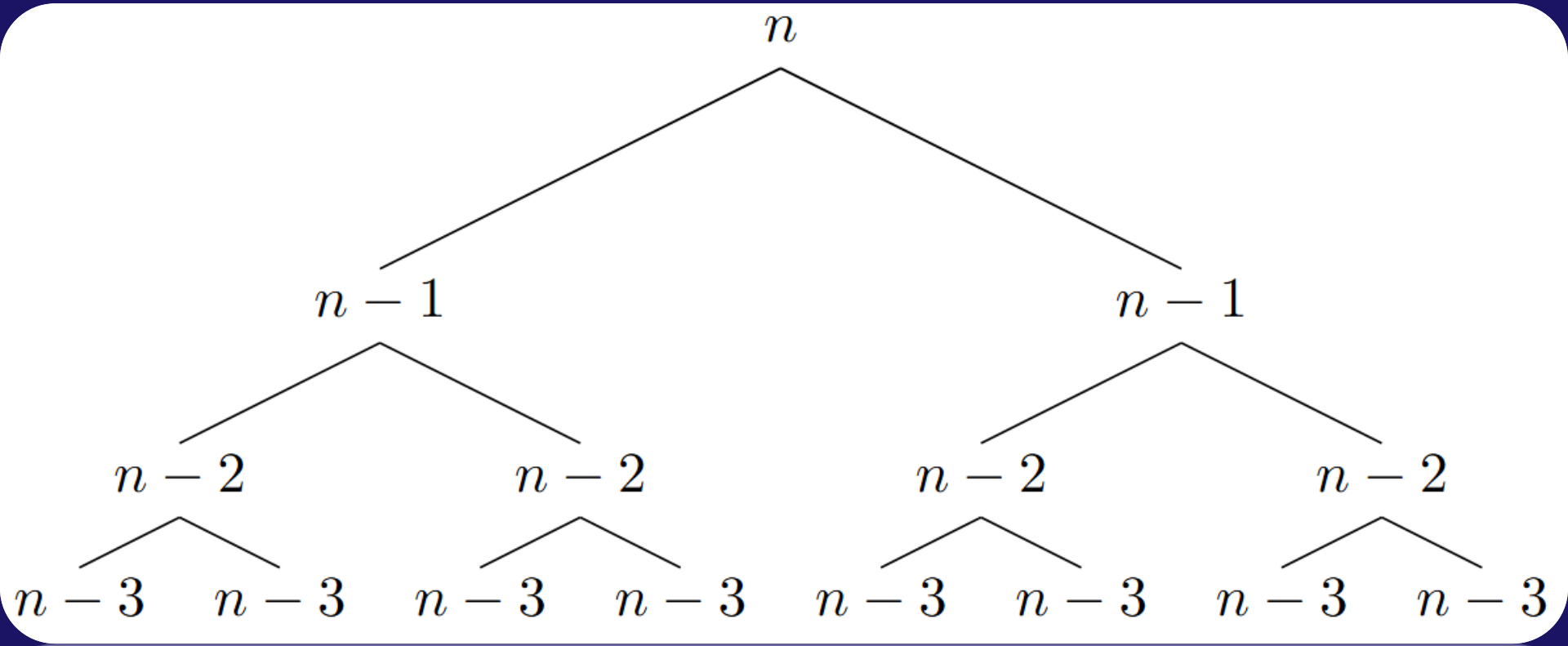
$$
\begin{aligned}
T(n) &= T(n-1) + T(n-2) + O(1) \\
&\leq T(n-1) + T(n-1) + O(1) \\
&= 2T(n-1) + O(1)
\end{aligned}
$$

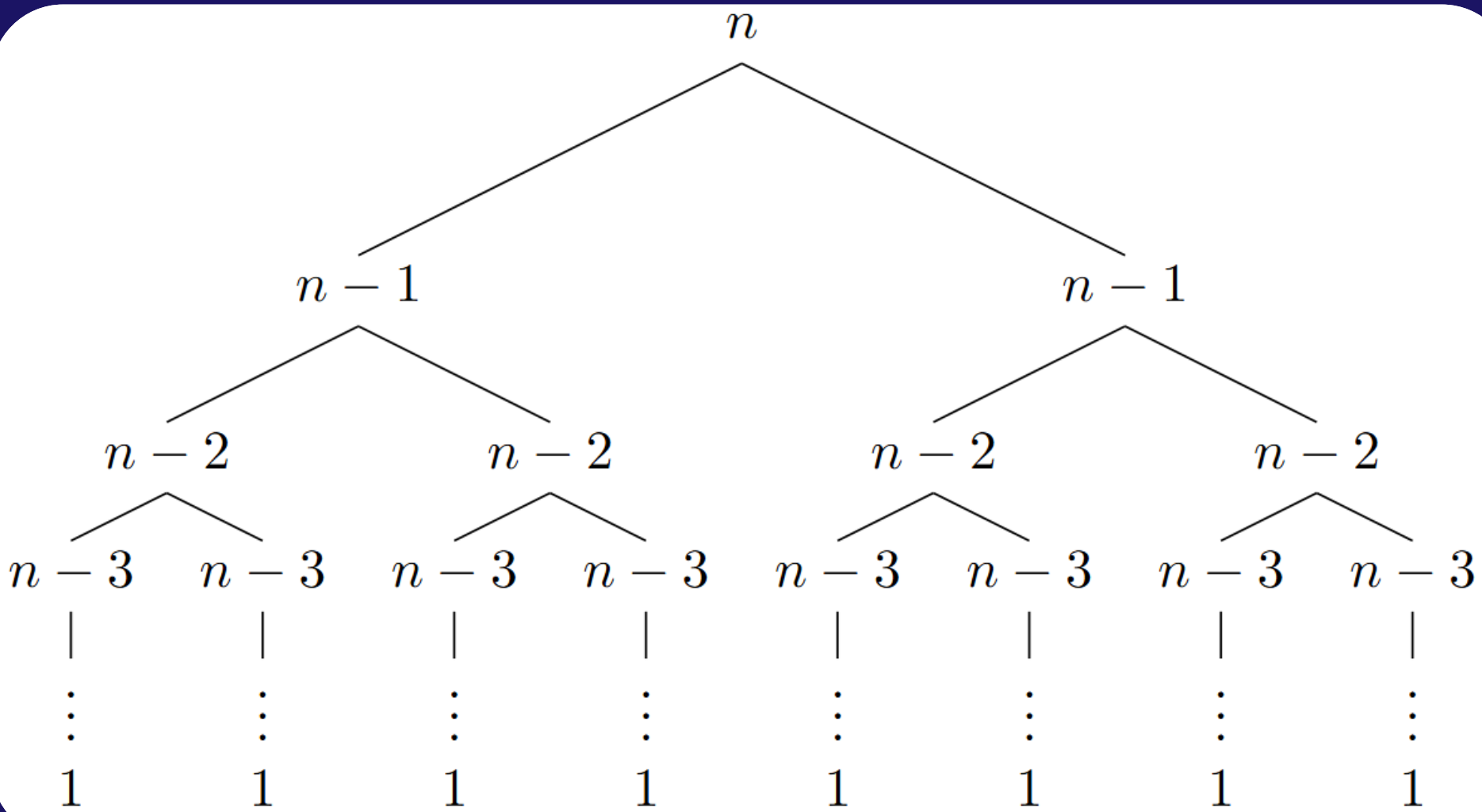
$$n$$


n

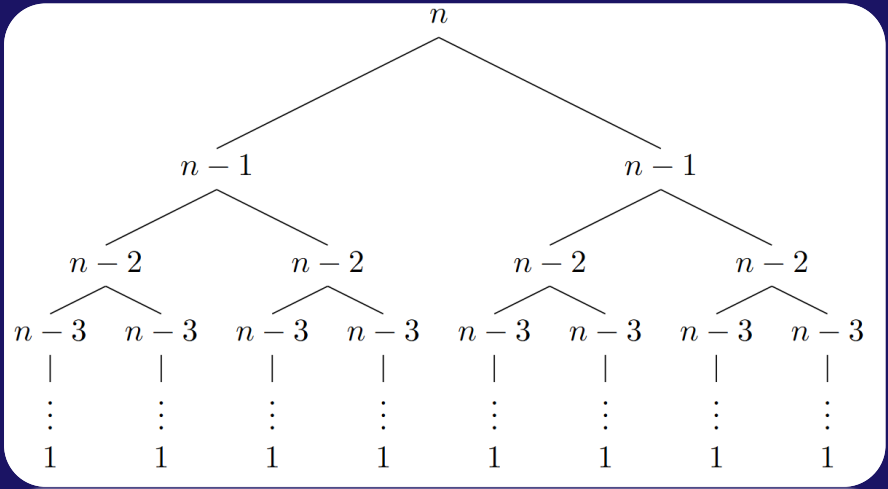

n
$n-1$
$n-1$


n
$n-1$
$n-1$
$n-2$
$n-2$
$n-2$
$n-2$


n
$n-1$
$n-1$
$n-2$
$n-2$
$n-2$
$n-2$
$n-3$
$n-3$
$n-3$
$n-3$
$n-3$
$n-3$
$n-3$
$n-3$

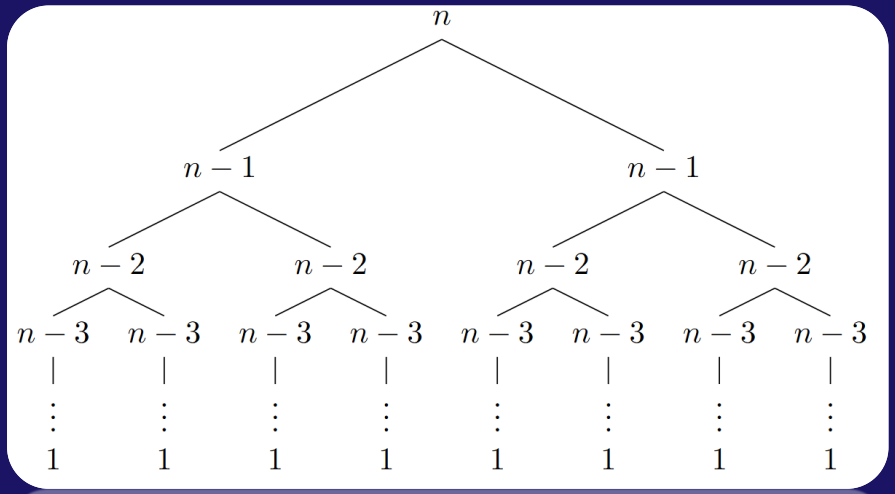

n
$n-1$
$n-1$
$n-2$
$n-2$
$n-2$
$n-2$
$n-3$
$n-3$
$n-3$
$n-3$
$n-3$
$n-3$
$n-3$
$n-3$
1
1
1
1
1
1
1
1

$$T(n) = 2T(n-1) + O(1)$$



1. Costo de un dato: c
2. Datos por nodo: $n - k$
3. Costo de nodo: $c\boldsymbol{f}(n-k) = c$
4. Nodos por nivel: 2^k

$$T(n) = 2T(n-1) + O(1)$$



5. Costo de nivel: $c(2^k)$

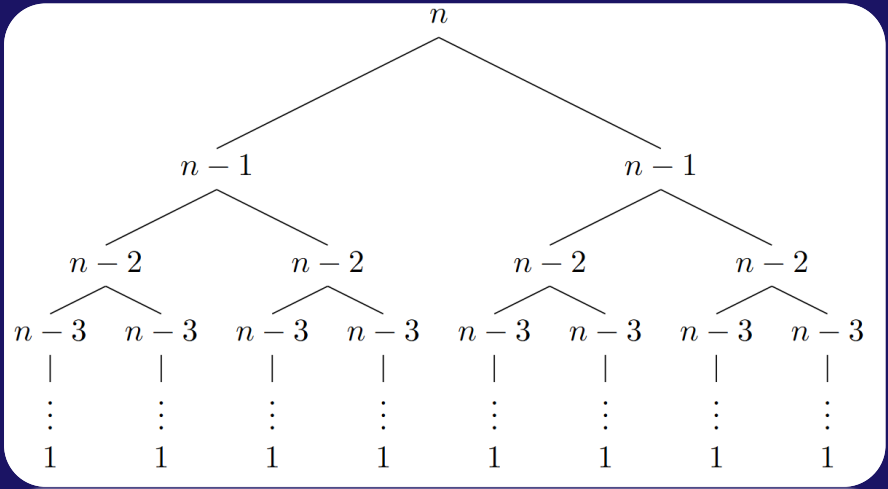
6. Costo de árbol:

$$\sum_{k=0}^{h} c(2^k) = c \sum_{k=0}^{h} 2^k$$

$$n - h = 1 \rightarrow h = n - 1$$

$$c \sum_{k=0}^{n-1} 2^k$$

$$T(n) = 2T(n-1) + O(1)$$



5. Costo de nivel: $c(2^k)$

6. Costo de árbol:

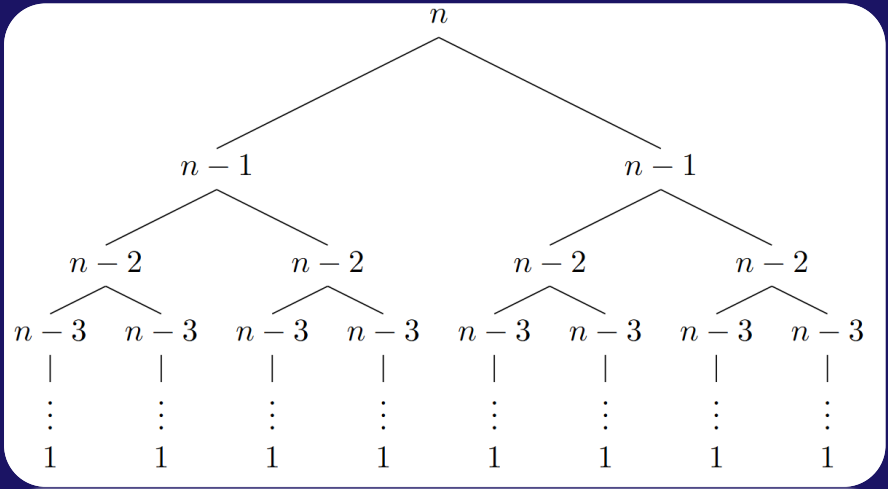
$$\sum_{k=0}^{h} c(2^k) = c \sum_{k=0}^{h} 2^k$$

$$n - h = 1 \rightarrow h = n - 1$$

$$c \sum_{k=0}^{n-1} 2^k$$

$$\sum_{k=0}^{n-1} ar^k = \frac{a(1-r^n)}{1-r}$$

$$T(n) = 2T(n-1) + O(1)$$



5. Costo de nivel: $c(2^k)$

6. Costo de árbol:

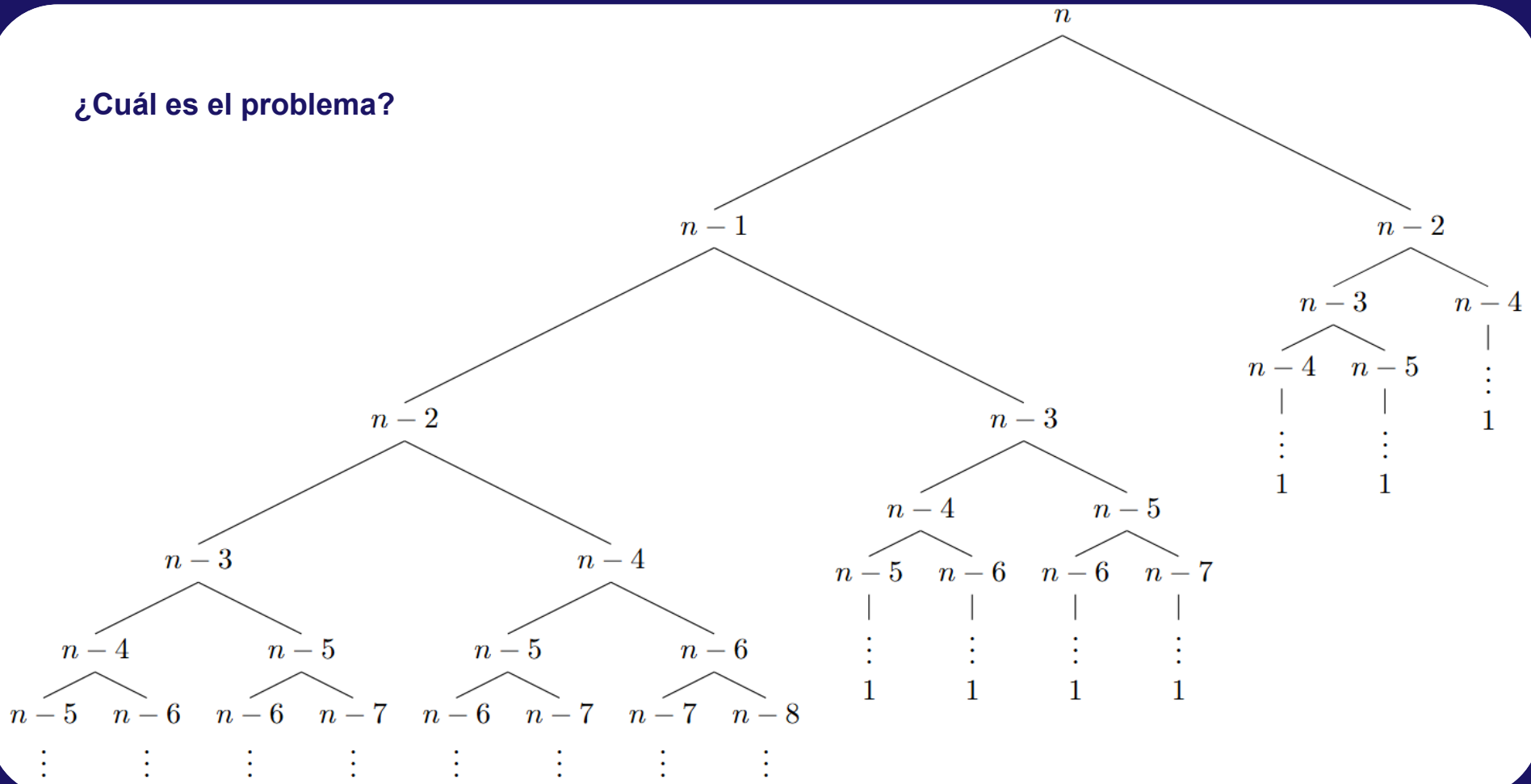
$$c\sum_{k=0}^{n-1} 2^k = c\frac{1-2^n}{1-2} = c(2^n - 1) = c(2^n) - c = O(2^n)$$

Contraste curioso:

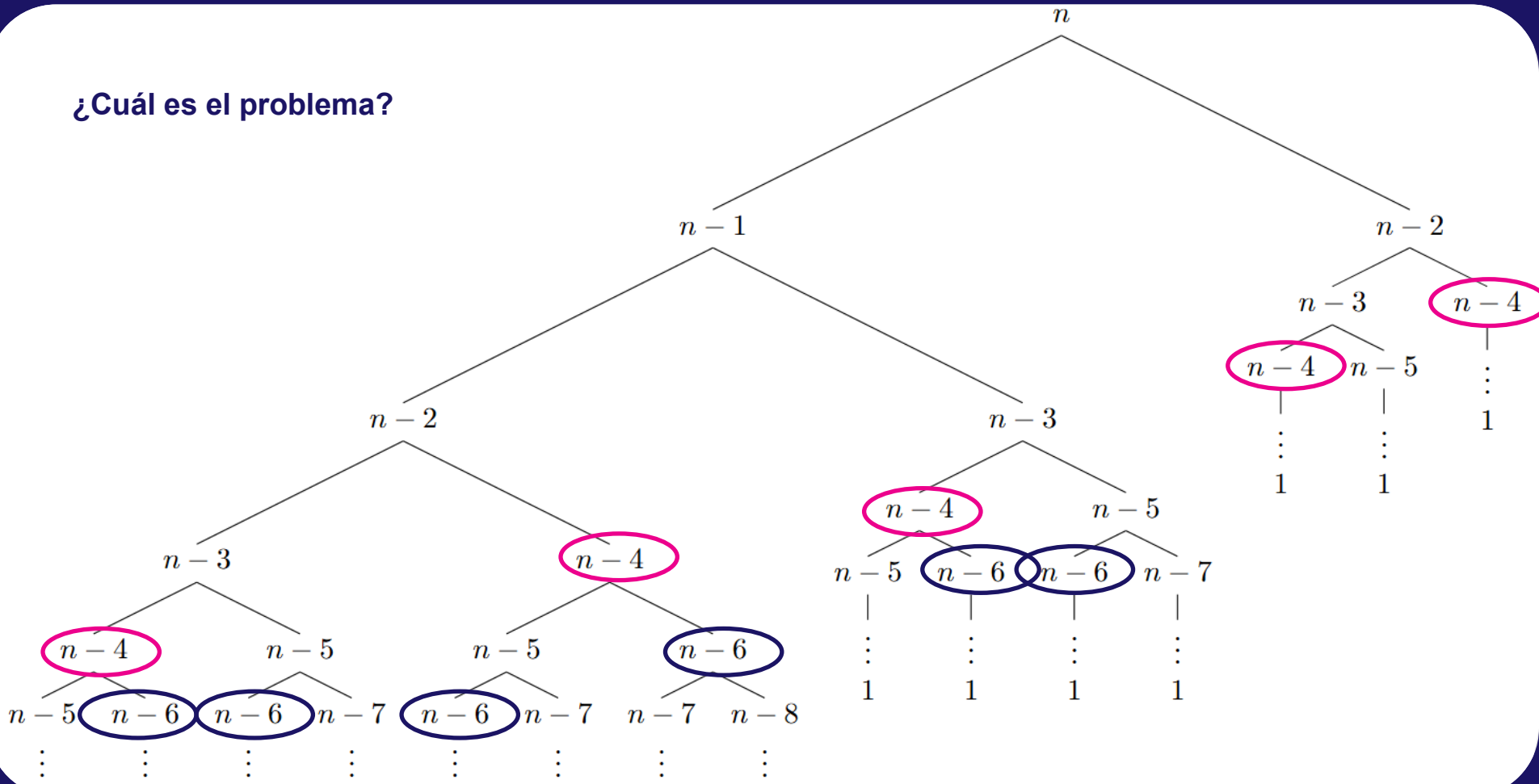
```
int factorial_posposicion(int n) {
    if( n == 0 || n == 1 ) return 1;
    return n * factorial_posposicion(n-1);
}
```

$$T(n) = O(n)$$

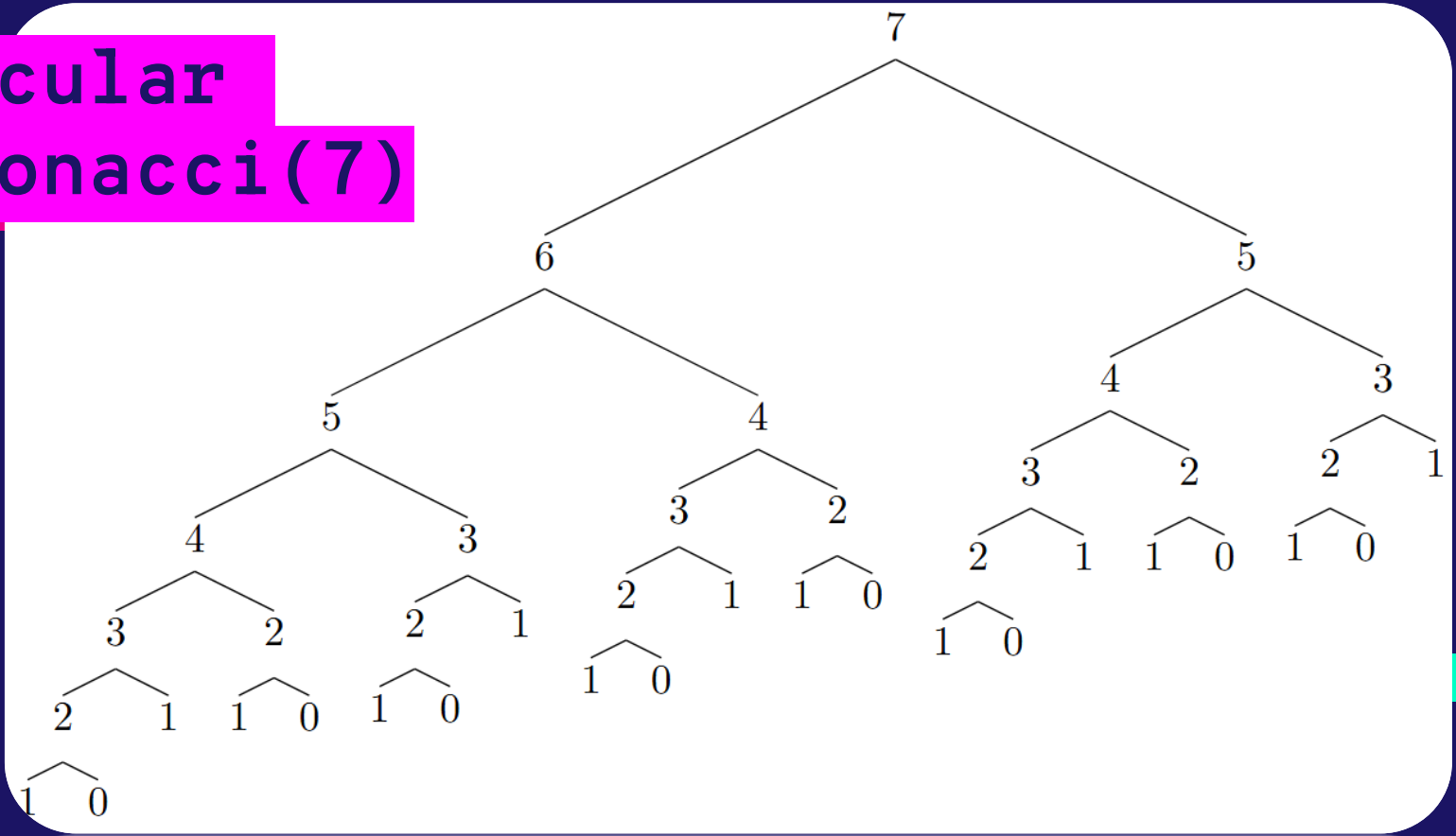
¿Cuál es el problema?



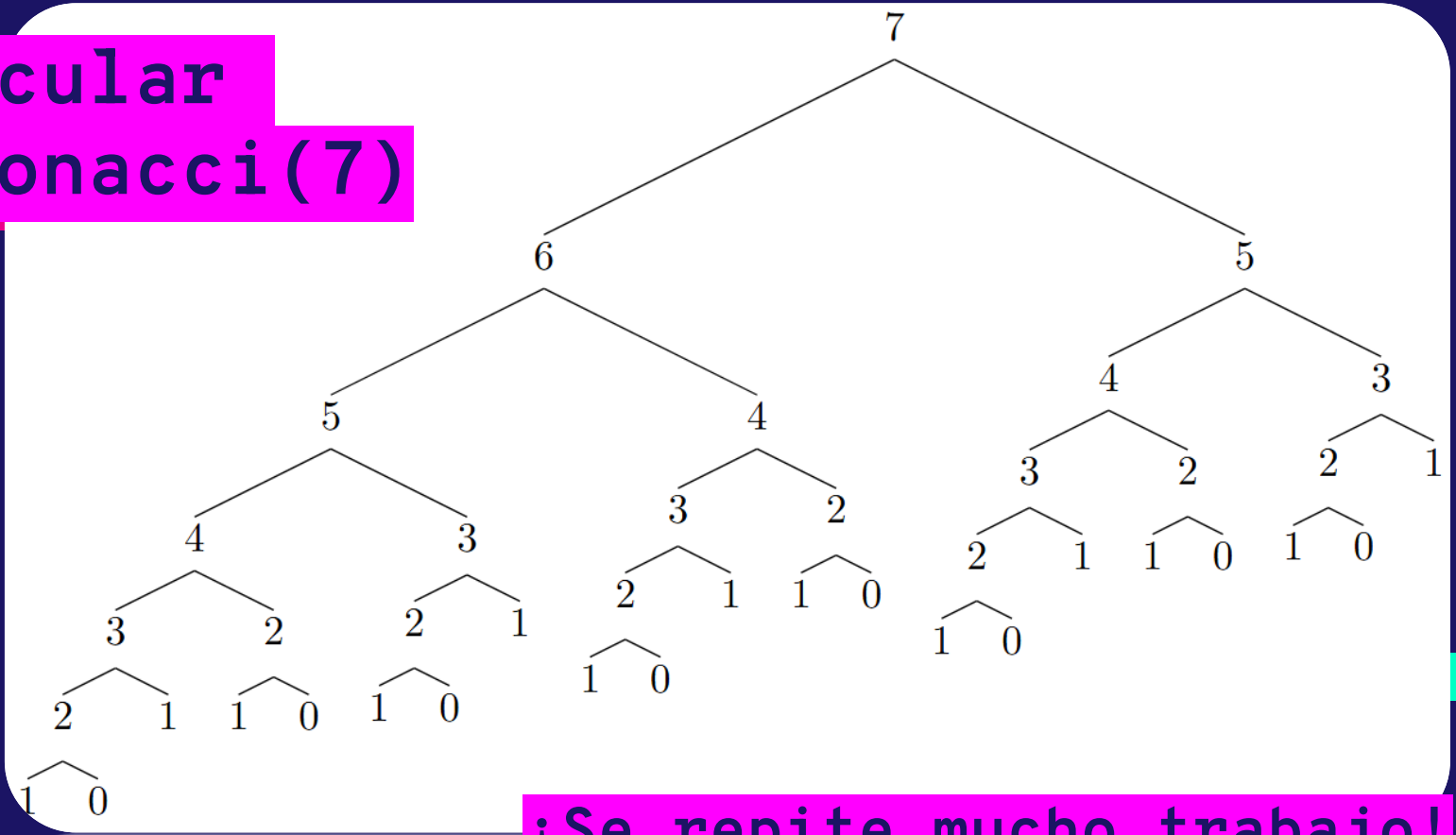
¿Cuál es el problema?



Calcular fibonacci(7)



Calcular fibonacci(7)



¡Se repite mucho trabajo!

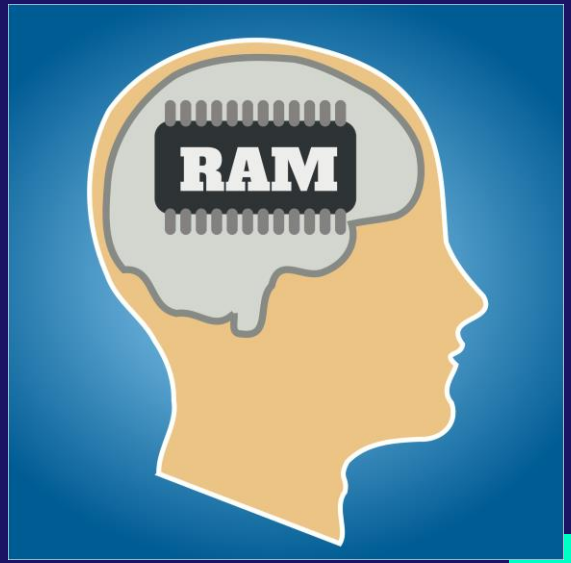
MEMOIZATION

Es una técnica de optimización utilizada para mejorar el rendimiento de los algoritmos al **almacenar los resultados de llamadas a funciones costosas** y reutilizarlos cuando vuelven a aparecer durante la ejecución.

Memoization

- La primera vez que una tarea ocurre:
  - Se calcula
  - Se almacena el resultado

- La segunda vez y veces posteriores:
  - Se reutiliza el resultado ya calculado



```
long fibonacci_memo(int pos) {
    if( pos == 0 ) return 0;
    if( pos == 1 ) return 1;

    vector<long> memo(pos+1,-1);
    memo[0] = 0;
    memo[1] = 1;

    return fibonacci_memo_aux(pos, memo);
}
```

```
long fibonacci_memo_aux(int pos, vector<long>& memo) {
    if( memo[pos] != -1 ) return memo[pos];
    memo[pos] = fibonacci_memo_aux(pos-1, memo) + fibonacci_memo_aux(pos-2, memo);
    return memo[pos];
}
```

Muchas Gracias

CREDITS: This presentation template was created by Slidesgo, incluiding icons by Flaticon, and infographics & images by Freepik.